

C++ - Tipos de Datos y Estructuras de Control

Patricio Olivares

Ingeniería Civil Telemática
Departamento de Electrónica
Universidad Técnica Federico Santa María

Estructura básica de un programa C++

```
In [ ]: #include <iostream>

int main(){
    std::cout << "Hola TEL102!" << std::endl;
    return 0;
}
```

- Inclusión de biblioteca(s) estándar
- Método **main**, encargado de la ejecución del programa

Ejercicio

Programa un algoritmo en C++ que sume dos valores: uno entero y uno real.

```
In [1]: #include <iostream>
int main(){
    int a = 3; // Variable entera
    float b; // Variable flotante
    b = 3.2;
    std::cout << a + b << std::endl; // 6.2
}
```

```
    return 0;
}
```

Tipos de datos primitivos

tipo	bytes	Rango
char	1	-127 to 127 or 0 to 255
unsigned char	1	0 to 255
signed char	1	-127 to 127
int	4	-2147483648 to 2147483647
unsigned int	4	0 to 4294967295
signed int	4	-2147483648 to 2147483647
short int	2	-32768 to 32767
unsigned short int	2	0 to 65,535
signed short int	2	-32768 to 32767
long int	4	-2,147,483,647 to 2,147,483,647
signed long int	4	same as long int
unsigned long int	4	0 to 4,294,967,295
float	4	+/- 3.4e +/- 38 (7 digits)
double	8	+/- 1.7e +/- 308 (15 digits)
long double	8	+/- 1.7e +/- 308 (15 digits)

Definición de variables

Definición, asignación e inicialización

```
In [2]: #include <iostream>

int main(){
    int a; // Definición variable a
    a = 3; // Asignación de un valor a la variable a

    int b = 5; // Inicialización de variable b (Definición + Asignación)

    std::cout << a + b << std::endl; // 8

    return 0;
}
```

Definición de variables

A diferencia de Python, C++ necesita definir explícitamente los tipos de cada variable (**tipificación explícita**).

```
In [6]: #include <iostream>
int main(){
    int a = 3; // Variable entera
    float b; // Variable flotante
    b = 3.2;
    std::cout << a + b << std::endl; // 6.2
    return 0;
}
```

Operadores

Aridad de operadores

- Unario (unitario): Un solo operando
- Binario: Dos operandos
- Ternario: Tres operandos

Expresiones Aritméticas - Operadores unarios

Operador	C/C++	Python
Negación lógica	!	not
Dirección	&	<i>No existe</i>
Derreferencia	*	<i>No existe</i>
Complemento a 1	~	~
Incremento	++	<i>No existe</i>
Decremento	--	<i>No existe</i>
Signo más	+	+
Negativo	-	-
Casting	(tipo)	<i>No existe</i>

Los operadores unarios agrupan de derecha a izquierda.
E.g. $-\sim x$ equivalente a $-(\sim x)$.

Expresiones Aritméticas - Operadores binarios

Operador	C/C++	Python
Suma	+	+
Resta	-	-
Multiplicación	*	*
División	/	/
División entera	/	//
Módulo	%	%
Exponenciación	<i>No existe</i>	**

División entera en C/C++ según tipo de dato.

Si tipo entero para numerador y denominador: división entera.

Expresiones Aritméticas - Operadores binarios

Operadores binarios-bitwise

Operadores a nivel de bit.

Operador	C/C++	Python
bitwise AND	&	&
bitwise OR		
bitwise XOR	^	^
Corrimiento izquierda	<<	<<
Corrimiento derecha	>>	>>

No son operadores aritméticos, pero pueden formar parte de expresiones aritméticas.

Expresiones Aritméticas - Precedencias

- **Precedencia:** Orden de evaluación de operadores adyacentes estableciendo prioridades
- **Asociatividad:** A igual precedencia, se define orden de evaluación
- **Paréntesis:** Altera y fuerza orden de evaluación

Expresiones Aritméticas - Precedencias

Precedencia	Descripción	Operador		Asociatividad
		C++	Python	
alta	exponente		**	←
	unarios (postfijos)	++ --		→
	unarios (prefijos)	++ -- ~ ! * & + -	~ + -	←
	casting de tipo	(type)		←
	multiplicativos	* / %	* / // %	→
	aditivos	+ -	+ -	→
	corrimiento	<< >>	<< >>	→
	bitwise AND	&	&	→
	bitwise XOR	^	^	→
	bitwise OR			→
baja	NOT lógico		not	←

Expresiones relacionales y booleanas

- **Expresión relacional:** Compara expresiones aritméticas. El valor resultante es un booleano. Incluye ==, !=, <, <=, >, >=
- **Expresión booleana:** Compara expresiones booleanas. El valor resultante es un booleano. Incluye AND, OR, NOT, y XOR

Expresiones relacionales y booleanas

Operadores Relacionales		
Operador	C/C++	Python
Igual	==	==
Distinto	!=	!= <>
Mayor	>	>
Menor	<	<
Mayor o igual	>=	>=
Menor o igual	<=	<=
Operadores Booleanos		
Operador	C/C++	Python
NOT	!	not
AND	&&	and
OR		or

Sentencias de asignación

- Permiten cambiar dinámicamente el valor ligado a una variable
- C/C++ permite incrustar una asignación en una expresión (como cualquier operador binario)

Sentencias de asignación

Operador	C/C++	Python
Asignación	=	=
Suma y asignación	+ =	+ =
Resta y asignación	- =	- =
Multiplicación y asignación	* =	* =
División y asignación	/ =	/ =
Módulo y asignación	% =	% =
Bitwise AND y asignación	& =	
Bitwise OR y asignación	=	
Bitwise XOR y asignación	^ =	
Corrimiento der. y asignación	>> =	
Corrimiento izq. y asignación	<< =	
Incremento	++	
Decremento	--	
División entera y asignación		// =
Exponente y asignación		** =

Sentencias de asignación - unarios de asignación

```
In [ ]: #include <iostream>

int main(){
    int contador = 0;
    int sum = ++contador;
    std::cout << sum << std::endl;
    return 0;
}
```

```
In [ ]: #include <iostream>

int main(){
    int contador = 0;
    int sum = contador++;
    std::cout << sum << std::endl;
    return 0;
}
```

Sentencias de asignación - operadores compuestos

```
In [ ]: #include <iostream>

int main(){
    int sum = 0;
    sum +=3; // sum = sum + 3;
    std::cout << sum << std::endl;
    return 0;
}
```

Sentencias de asignación - operador ternario

retorno = op1 ? op2 : op3;

- Retorna **op2** si **op1** es verdadero, sino, retorna **op3**

```
In [ ]: #include <iostream>

int main(){
    int a = 10;
    int cuenta = (a > 0) ? 0 : 1; // C/C++
    (a>10) ? a : cuenta = 200; // C++
    std::cout << cuenta << std::endl;

    return 0;
}
```

Conversión de tipos

- **Coerción:** Conversión automática de tipos
- **Casting:** Conversión definida por el programador de tipos

```
In [ ]: int main(){
    int pi_int = 3.1415; // coerción
    std::cout << pi_int << std::endl;
    float pi_float = 3.1415;
    pi_int = (int) pi_float; // casting
    std::cout << pi_int << std::endl;

    return 0;
}
```


Ejemplo

Desarrolle un programa que permita transformar temperaturas desde grados Fahrenheit a Celsius.

Estructuras de control

- Permiten alterar la secuencia de ejecución de un programa
- Normalmente existen dos tipos
 - **Selección:** múltiples alternativas de ejecución, controladas por una condición
 - **Iteración:** Ejecución repetitiva de un grupo de sentencias también controladas por una condición
- Para agrupación de sentencias sobre las cuales se ejerce el control, en C/C++ se usa las llaves {} (en Python se hace por indentación)

Estructuras de control: if - else if - else

Desarrolle un programa en C++ que solicite un número por entrada estándar y lo clasifique según su signo.

```
In [ ]: #include <iostream>
```

```
int main(){
    int x;

    std::cin >> x;

    if(x<0){
        std::cout << "Este es un número negativo" << std::endl;
    }else if(x==0){
        std::cout << "Este número es igual a cero" << std::endl;
    }else{
        std::cout << "Este es un número positivo" << std::endl;
    }

    return 0;
}
```

Estructuras de control: if - else if - else

```
if(condicion_1){
    // código
    // ...
}
```

```

} else if(condicion_k){
    // más código
} else {
    // mucho más código
}

```

Corto circuito

- **Corto circuito:** Evaluación de una expresión evitando ejecuciones innecesarias
- C/C++ definen corto circuito para **&&** y **||**

```

In [ ]: #include <iostream>

int main(){
    int x = 1;

    if(x==1 || x++) { //|| -> or de python, && -> and de python
        std::cout << "El valor de x es " << x << std::endl;
    }

    return 0;
}

```

Estructuras de control: switch

Desarrolle un programa en C++ que solicite un número entero y diga si es 1, 2, 3 u otro número.

```

In [ ]: #include <iostream>

int main(){
    int x;
    std::cin >> x;

    switch(x){
        case 1:
            std::cout << "x vale 1" << std::endl;
            break;
        case 2:
            std::cout << "x vale 2" << std::endl;
            break;
        case 3:
            std::cout << "x vale 3" << std::endl;
            break;
        default:
            std::cout << "x no vale ni 1, ni 2 ni 3" << std::endl;
    }
}

```

```
    return 0;
} // ¿Qué ocurre si no escribimos break?
```

Estructuras de control: switch

```
switch(x){ // x debe ser una variable
    case x1: // x1 es un valor a comparar
        // código
        break;
    case x2:
        // código
        break;
    //...
    case xN:
        // código
        break;
    default:
        // código
}
```

Estructuras de control: while - do while

Desarrolle un programa en C++ que solicite un número entero y cuente desde 1 hasta éste.

```
In [ ]: #include <iostream>

int main(){
    int i = 1, num;
    std::cin >> num;

    while(i <= num){
        std::cout << i << std::endl;
        i++;
    }

    return 0;
}
```

```
In [ ]: #include <iostream>

int main(){
    int i = 1, num;
    std::cin >> num;

    do{
        std::cout << i << std::endl;
        i++;
    }while(i<=num);
}
```

```
    return 0;
}
```

Estructuras de control: while - do while

```
while(condicion){
    // código
}

do{
    // código
}while(condicion);
```

Estructuras de control: while - do while

```
In [ ]: #include <iostream>

int main(){
    int i = 3;

    while(i < 3){
        std::cout << i << std::endl;
        i++;
    }

    return 0;
}
```

```
In [ ]: #include <iostream>

int main(){

    int i = 3;

    do{
        std::cout << i << std::endl;
        i++;
    }while(i<3);

    return 0;
}
```

Estructuras de control: for

Desarrolle un programa en C++ que cuente desde 1 hasta diez.

```
In [ ]: #include <iostream>
```

```
int main(){
    for(int i = 1; i <= 10; i++){
        std::cout << "El valor de i es " << i << std::endl;
    }

    return 0;
}
```

Estructuras de control: for

```
for (inicializacion; condicion_de_termino; actualizacion){
    // código
}
```

Estructuras de control: break - continue - return

- **break:** Permite salir del ciclo más cercano
- **continue:** Permite pasar al ciclo siguiente
- **return:** Permite, desde una función, retornar el control a la sentencia que hizo la invocación

Estructuras de control: break - continue - return

```
In [ ]: #include <iostream>

int main(){
    int n;
    do {
        std::cout << "Ingrese numero:";
        std::cin >> n;
        if (n<0)
            break;
        if (n>10) {
            std::cout << "Saltarse el valor." << std::endl;
            continue;
        }
        std::cout << "El numero es:" << n << std::endl;
    } while (n!= 0);

    return 0;
}
```

Subprogramas

- Permiten encapsular y reutilizar código
- Normalmente requieren memoria dinámica de stack
- Tipos de subprogramas:
 - Procedimientos o subrutinas (sin valor de retorno)
 - Funciones (con valor de retorno)
 - Métodos y constructores en Orientación a Objetos

Subprogramas: funciones

Se puede separar el prototipo de la implementación

- **Prototipo:** Solo define la existencia y formato de la función. Suficiente para ser invocada.

```
In [ ]: float potencia(float, float);
```

Subprogramas: funciones

- **Implementación:** Define el código de dicha función.

```
In [ ]: float potencia(float base, float exp){
        float result = 1;
        while (exp != 0) {
            result *= base;
            --exp;
        }

        return result;
    }
```

```
In [ ]: #include <iostream>

int main(){
    std::cout << potencia(2,3) << std::endl;
    return 0;
}
```

Tipos de datos estructurados - Arreglos

- Consiste en una colección homogénea y ordenada de elementos.
- Se identifican por su posición relativa mediante un *índice* encerrado entre paréntesis cuadrados.
- El índice parte en 0.

```
In [ ]: #include <iostream>

int main(){
    int arreglo[10]; // Arreglo estático de 10 elementos enteros
    for(int i=0; i<10; i++){ // for para recorrer arreglo usando índice
        arreglo[i] = i*i; // Asignación de cada posición con un valor
    }

    for(int j=0; j<10; j++){ // Mostrar elementos en el arreglo
        std::cout << "Elemento en la posición " << j << " es " << arreglo[j]
    }

    return 0;
}
```

Tipos de datos estructurados - Arreglo de caracteres

- Las cadenas de caracteres (string) se definen en C como arreglos de tipo char.
- El término de la cadena queda definida con el caracter '\0'.
- Para manejo de strings se utiliza la biblioteca **<cstring>**

```
In [ ]: #include <iostream>
#include <cstring>

int main(){

    char str1[10] = "hola ";
    char str2[10] = "mundo";
    std::cout << strlen(str1) << std::endl;
    std::cout << strcat(str1, str2) << std::endl;
    strcpy(str1, "chao");
    std::cout << str1 << std::endl;

    return 0;
}
```

Tipos de datos abstractos - Estructuras

- Conjunto heterogéneo de datos (ej. estructuras y clases).

- Cada elemento individual (campo, miembro o atributo) es identificado con un nombre.
- Permite encapsular información.

```
In [ ]: #include <iostream>
#include <cstring>

struct empleado{
    char nombre[40];
    float sueldo;
};

int main(){
    struct empleado e1;
    e1.sueldo = 5000;
    strcpy(e1.nombre, "Juancho");

    std::cout << "El empleado es " << e1.nombre << " y su sueldo es " << e1.

    return 0;
}
```